Autonomous Earthen Inspection: Final Experiment Execution Guide

Drone Inspection Team

September 1, 2026

1 Overview

This document outlines the complete procedure to replicate the empirical study for the autonomous earthen inspection drone. The execution relies on a hardware-accelerated Docker Compose stack leveraging an NVIDIA GPU to run both the 3D physics simulation (Gazebo Harmonic) and the real-time AI object detector (YOLOv11s) in parallel.

2 Prerequisite: NVIDIA GPU Toolkit

Because the evaluation requires sub-10ms inference speeds to test the confidence-triggered revisit strategy, the host Ubuntu machine **must** have an NVIDIA GPU and the NVIDIA Container Toolkit installed. Ensure your system is ready by verifying you can run `nvidia-smi` inside a generic Docker container.

3 Required Asset Placement

Before executing the pipeline, the trained AI weights and the 3D world must be injected into the repository. The provided master script strictly expects them at these precise relative locations:

3.1 YOLOv11s Weights

Copy the PyTorch `.pt` file generated from the 50-epoch earthen dataset training to:

```
models/yolo/yolo_earthen_v11.pt
```

3.2 Custom 3D Map (Earthen Houses)

Copy the `.glb` mesh containing the structural defects to:

```
gazebo_simulation/worlds/custom_map.glb
```

Note: The simulation relies on a custom `custom_inspection.sdf` *Gazebo world file which is programmed to dynamically import this GLB mesh at origin* $(0, 0, 0)$.

4 Execution Orchestration

Once the assets are in place, the entire architecture (PX4 flight controller, MAVROS, A* Planner, OctoMap server, and the ROS2 AI pipeline) is automated.

From the root of the repository, execute the following command:

```
bash scripts/run_final_article_experiment.sh
```

Under the hood, the master script autonomous flow:

1. Spins up `uas_sim_headless` and `uas_ai_gpu` Docker containers.
2. Initializes the PX4 EKF2 state estimator and OctoMap volumetric mapping.
3. Launches the YOLOv11s inspection node and the confidence-triggered Revisit generator via ROS2.
4. Dispatches the initial spatial coverage waypoints to the drone.
5. Records all relevant ROS topics (Telemetry, Depth mapping, Detection bounding boxes) to a time-stamped rosbag.

5 Result Extraction

When the drone finishes the coverage path and any dynamically generated revisit waypoints, the script will gracefully terminate the simulation to finalize the rosbag metadata.

It will then automatically run the offline analysis suite. You can find the required figures and metrics for the final article in:

```
rosbags/<latest_run_timestamp>/analysis/
```

This folder will contain:

- `trajectory.png` (Top-down visual plot of the flight path and dynamic revisits)
- `local_position.csv` & `velocity_local.csv` (For drone pose analysis)
- `gps.csv` & `battery.csv`
- Extracted MP4 videos highlighting the detected structural defects.